

MiniTP de ARM

Resolución de problemas sobre datos en memoria

Orga, Com7

Objetivos

En este trabajo práctico se busca que el/la estudiante ejercite:

- Acceso a datos ubicados en memoria (direccionamiento);
- Uso de instrucciones de movimiento de datos y aritméticas;
- Comparación de valores y control de flujo;
- Uso de subrutinas;
- Recorrido de cadenas de caracteres en memoria;
- Distinción entre **valor numérico** y **codificación ASCII**.

Condiciones generales

1. El trabajo debe resolverse en **lenguaje ensamblador ARM 32 bits para Rpi**.
2. **No es necesario** el uso de interrupciones, llamadas al sistema, rutinas de biblioteca, ni mecanismos de entrada/salida por pantalla, teclado o archivos.
3. La verificación del funcionamiento se realizará **observando registros y posiciones de memoria** mediante el debugger.
4. Los datos de entrada estarán cargados en memoria mediante etiquetas, y el programa deberá dejar los resultados también en memoria.
5. El programa debe estar escrito de manera **general**, es decir, **no debe depender de un único juego de datos**. Durante la corrección se podrán modificar los valores almacenados en memoria.
6. El código debe estar **comentado** y organizado de forma clara.

Ejercicio 1 – Calculadora

Enunciado

Se tienen en memoria dos operandos enteros y un tercer dato que contiene un carácter ASCII correspondiente a una operación aritmética. El programa deberá leer ese carácter y, según su valor, realizar la operación indicada entre los dos operandos.

Los operadores posibles son:

- '+' suma
- '-' resta
- '*' multiplicación
- '/' división entera

El resultado de la operación deberá quedar almacenado en memoria.

Datos de entrada

Se dispondrá en memoria de tres etiquetas:

- una etiqueta con el primer operando;
- una etiqueta con el segundo operando;
- una etiqueta de un byte con el operador ASCII.

Por ejemplo:

op1:	.word	25
op2:	.word	7
operador:	.byte	'+'

Salidas esperadas

El programa deberá dejar en memoria:

- el resultado de la operación;
- un código de estado que indique si la operación se realizó correctamente o si ocurrió un error.

Por ejemplo:

resultado:	.word	0
estado:	.word	0

Códigos de estado sugeridos

Se sugiere utilizar:

- 0 → operación realizada correctamente
- 1 → operador inválido
- 2 → división por cero

Si se desea, estos códigos pueden modificarse, pero deben estar claramente documentados.

Requisitos específicos

1. **Debe utilizarse al menos una subrutina por operación.** Es decir, el programa principal debe decidir qué operación corresponde y luego invocar la subrutina adecuada.
2. Debe existir una rutina principal que:
 - lea los datos desde memoria;
 - compare el operador con los caracteres ASCII válidos;
 - invoque la subrutina correspondiente;
 - almacene el resultado y el estado final en memoria.
3. En caso de encontrar un operador no válido, el programa **no debe ejecutar una operación incorrecta**: debe informar el error mediante el código de estado.
4. En caso de división por cero, el programa **no debe intentar dividir**: debe dejar indicado el error correspondiente en memoria.
5. El trabajo debe resolver el problema **para cualquier valor de operandos** que respete el rango de representación utilizado.

Aclaraciones importantes

- Se evaluará especialmente la **separación en subrutinas**, el uso correcto de saltos y retornos, y la claridad de la solución.

Dificultades inherentes del problema

Este ejercicio presenta varias dificultades que deben ser comprendidas antes de comenzar:

1. **Separar dato y operación.** El programa no conoce de antemano qué operación debe hacer: debe leer un carácter desde memoria, interpretarlo y recién entonces decidir el camino de ejecución.
2. **Trabajar con caracteres ASCII.** El operador no está guardado como una palabra como *suma* o *resta*, sino como un byte que contiene el código ASCII de un símbolo. Por lo tanto, el programa debe hacer comparaciones explícitas contra esos caracteres.
3. **Resolver un mismo problema con varios caminos posibles.** No alcanza con escribir una única secuencia aritmética: el programa debe poder elegir entre varios comportamientos posibles según el operador leído.
4. **Uso de subrutinas.** La división del problema en subrutinas obliga a organizar el código, controlar correctamente las llamadas y retornos, y decidir qué registros se usan para pasar datos y devolver resultados.
5. **Manejo de errores sin entrada/salida.** Como no se puede imprimir mensajes por pantalla, los errores deben informarse mediante valores en memoria. Esto obliga a pensar cómo representar el estado final del programa.

Ejercicio 2 – Conversión de una cadena ASCII a entero

Enunciado

Se tiene en memoria una cadena de caracteres que representa un número decimal no negativo, por ejemplo:

```
cadena: .asciz "1547"
```

El programa deberá recorrer la cadena y convertirla al valor numérico correspondiente, dejando el resultado en memoria como entero.

En el ejemplo dado, el programa deberá obtener el valor numérico 1547 y almacenarlo como palabra en memoria.

Datos de entrada

Se dispondrá en memoria de una cadena **ASCIIZ**, es decir, una secuencia de caracteres ASCII terminada en byte cero.

Ejemplo:

```
cadena:      .asciz "1547"
```

Salidas esperadas

El programa deberá dejar en memoria:

- el valor numérico obtenido;

Por ejemplo:

```
numero:      .word    0
```

Pistas

Primero intentar resolverlo por cuenta propia, si no sale, aquí hay algunas ayudas

1. El programa debe recorrer la cadena **carácter por carácter** hasta encontrar el byte nulo de terminación.
2. Cada carácter leído debe convertirse de ASCII a su valor numérico real.
3. La acumulación del resultado debe realizarse siguiendo la lógica del sistema decimal posicional. En cada paso:

$$\text{acumulador} = \text{acumulador} \times 10 + \text{dígito}$$

1. El valor final obtenido debe almacenarse en memoria.

Aclaraciones importantes

- La cadena contiene **caracteres**, no números binarios ya convertidos.
- Por ejemplo, el carácter '1' tiene código ASCII decimal 49 (0x31), y no el valor numérico 1.
- Para obtener el valor de un dígito decimal a partir de su carácter ASCII, debe realizarse la conversión correspondiente.

- El programa debe ser general: no se acepta una solución que cargue directamente el valor final esperado.
- La cadena será **ASCIIZ**: el recorrido finaliza al encontrar el byte 0.

Dificultades inherentes del problema

Este ejercicio presenta dificultades conceptuales y de implementación que deben tenerse en cuenta:

1. **Distinguir carácter y número.** La cadena "1547" no contiene el número 1547 en binario, sino los caracteres '1', '5', '4' y '7', cada uno codificado en ASCII.
2. **Conversión de cada símbolo.** No basta con copiar bytes: cada carácter debe transformarse en su dígito correspondiente.
3. **Construcción progresiva del resultado.** El número no se obtiene sumando dígitos, sino respetando la notación posicional decimal. Por eso, en cada paso hay que multiplicar el acumulador por 10 y luego sumar el nuevo dígito.
4. **Recorrido de cadenas en memoria.** El programa debe avanzar byte a byte hasta encontrar el terminador nulo. Esto obliga a controlar correctamente punteros, offsets o modos de direccionamiento.
5. **Validación de entrada.** Si la cadena contiene un carácter no numérico, el programa debe detectarlo. Como no hay salida por pantalla, esa situación debe reflejarse mediante un código de estado en memoria.

Entrega

La entrega deberá incluir:

1. **Archivo fuente en ensamblador ARM** con la resolución de ambos ejercicios.
2. **Breve informe** en PDF o en texto, donde se explique:
 - cómo está organizado el programa;
 - qué subrutinas fueron implementadas;
 - qué registros se utilizan de entrada y salida, si corresponde;
 - al menos un juego de prueba para cada ejercicio.

Criterios y Condiciones de entrega

- La entrega es grupal;
- Este MiniTP se entregará dos veces en las fechas indicadas por Moodle;
- correcta implementación del control de flujo;
- uso adecuado de **subrutinas** en el Ejercicio 1 (no realizar saltos condicionales);
- claridad, prolijidad y comentarios del código;
- generalidad de la solución.
- La entrega es obligatoria para regularizar la cursada
- La entrega es obligatoria para regularizar la cursada

Observaciones finales

- La cátedra podrá modificar los valores de entrada durante la corrección.
- No se aceptarán soluciones que dependan de un único caso particular.
- No se permite resolver el problema cargando directamente el resultado esperado.
- Se recomienda probar el funcionamiento con distintos datos desde el debugger.
- No resolver con agentes de IA para generación de código.